

Intel SGX 硬件安全计算系统大图

L0 Essence:

[M1.1] 处理器预留内存（PRM）与飞地页面缓存（EPC）分配

\\textbf{P}: BIOS 在启动阶段通过 PRMRR 寄存器划分出一块不受任何 CPU 之外设备（包括 DMA）访问的 Processor Reserved Memory (PRM), 并在其中切分出 Enclave Page Cache (EPC) 用以分配物理 Enclave 页面。\\textbf{D}: 物理 EPC 页面由内核中的 SGX 驱动维护。在 Linux 5.11+ 中，由核心函数 `sgx_alloc_epc_page` 分配，页面大小为 4KB。PRM 最大支持几百 MB，耗尽时会触发软件分页。\\textbf{S}: `main.c`:L30-80

[M1.2] 飞地页面缓存表（EPCM）硬件成员访问控制规则

\\textbf{P}: EPCM 是 CPU 内部维护的安全元数据表，包含了每个 EPC 页面的权限（R/W/X）、类型（SECS, REG, TCS, VA）及所属 Enclave (SECS ID) 等，硬件层在 MMU 地址翻译后强制比对 EPCM。\\textbf{D}: 即使内核页表将某 EPC 页面的物理地址映射给了普通进程，CPU 执行单元检测到当前不处于该 Enclave 内部执行状态时，也会抛出 #GP 异常，完全绕过了 OS 内核。\\textbf{S}: `encl.c`:L120-160

[M1.3] 内存加密引擎（MEE）的实时 AES-MCTS 加密与重放保护

\\textbf{P}: MEE 是嵌入在 CPU 内存控制器内部的专用硬件。当数据从 CPU 缓存写入 PRM 时，MEE 采用 AES-XTS 模式对其进行加密；读入缓存时解密。\\textbf{D}: 为防止重放攻击和剪切攻击，MEE 内部维护了一棵基于 Merkle-Tree 的 MCTS (Memory Cryptographic Tree)，根节点常驻 CPU，从而对物理总线窃听者呈现随机噪音。\\textbf{S}: `main.c`:L180-210

[M1.4] 飞地硬件生命周期指令（ECREATE/EADD/EINIT/EEXTEND）

\\textbf{P}: 构建飞地包括：ECREATE 创建 SECS 安全上下文，EADD 将代码/数据页面加载进 EPC，EEXTEND 对页面 256 字节分块进行累加度量，EINIT 锁定飞地并锁定度量寄存器。\\textbf{D}: 运行 EINIT 时，需要传入包含 Launch Token 的 SIGSTRUCT 签名包。如果平台启动密钥配置不合规，飞地将初始化失败。\\textbf{S}: `ioctl.c`:L200-290

[M1.5] Enclave 模式下 CPU 环 3 执行限制与内存边界

\\textbf{P}: 进入 Enclave 内部后，CPU 强制处于特殊的“Enclave 模式”，且仅允许在 Ring 3 执行，禁止执行大多数系统敏感特权指令（如 IN, OUT, INT, CLI, STI）。\\textbf{D}: 在 Enclave 模式下发生任何 I/O 操作、特权级变动或外部中断，CPU 都会发生 AEX，切出飞地，以防止飞地内部越权或破坏宿主系统稳定。\\textbf{S}: `encl.c`:L300-340

[M2.1] 异步飞地退出（AEX）时寄存器状态清空与恢复机制

\\textbf{P}: 当 Enclave 内部发生中断或硬件异常时，CPU 不会将上下文直接交给普通异常处理程序（防止信息泄露），而是执行 AEX，自动用零或哑元数据覆盖全部通用寄存器。\\textbf{D}: 真实的 CPU 状态被硬件压入飞地内部的 State Save Area (SSA) 栈帧。异常处理结束后，通过 ERESUME 指令恢复 SSA 寄存器状态并重新进入 Enclave。\\textbf{S}: `assembly.S`:L10-60

[M2.2] 飞地页错误（SGX #PF）与控制通道时序侧信道防御

\\textbf{P}: OS 依然负责 Enclave 内部虚拟地址到物理 EPC 页的页表映射。如果发生换页，会触发页错误。恶意 OS 可以故意构造频发的页错误来分析 Enclave 的数据访问控制流。\\textbf{D}: 防御方案包括：在 Enclave 内部检测时钟周期比率变化，或使用软件层自适应平滑路径（如 ORAM）抹平内存访问的时序差异。\\textbf{S}: `encl.c`:L410-450

[M2.3] 瞬态执行漏洞（L1TF / Foreshadow）在 TEE 中的防御与缓解

\\textbf{P}: Foreshadow 漏洞利用了 CPU 乱序执行的分支预测机制。即使页表呈现 Present = 0，CPU 预测执行依然可能将 L1 Cache 中的 EPC 数据泄露给推测代码。\\textbf{D}: 缓解方式是关闭超线程（Hyper-Threading），强制刷新 L1 Data Cache，或在微码层面隔离宿主与飞地的物理核绑定关系。\\textbf{S}: `main.c`:L310-350

[M2.4] 线程控制结构（TCS）与状态保存区（SSA）帧设计

\\textbf{P}: TCS 是飞地内部固定的硬件页面，包含飞地线程的入口地址（OENTRY）、SSA 栈的基址及偏移，每个逻辑 CPU 在飞地中必须独占一个 TCS。\\textbf{D}: SSA 包含了保存通用寄存器值、浮点数及 SSE 状态所需的物理缓存空间。TCS 与 SSA 物理页面必须在 EINIT 之前通过 EADD 被加载且设为不可变。\\textbf{S}: `encl.h`:L50-80

[M2.5] SGX 飞地页面换出（EBLOCK/ETRACK/EWB）与硬件重加密

\\textbf{P}: 当物理 EPC 页不够用时，操作系统必须配合 CPU 将飞地页面换出（Paging）到不安全的主存。这需要调用 EBLOCK 阻塞访问，ETRACK 清理页表的 TLB。\\textbf{D}: 接着运行 EWB 将页面移出，期间 CPU 会使用临时封包密钥（Wrap Key）对页面加密并生成版本令牌，写回普通内存；换入时执行 ELDU 重新载入解密并比对版本令牌。\\textbf{S}: `ioctl.c`:L340-410

L1 Logical Framework

L2 Data Flow Topology

Trade-off Matrix: Security vs Performance

[M3.1] 本地认证硬件指令（EReport/EGETKEY）及 symmetric 共享密钥

\\textbf{P}: 同一平台上两个飞地之间要建立信任，可调用 EREPORT 生成一份包含对方 SECS 度量的 Report，并在 CPU 内部采用对称密钥（Report Key）生成 MAC。\\textbf{D}: 对方飞地收到 Report 后，调用 EGETKEY 获取对应的 Report Key，并重新计算 MAC，一致则代表两者均在同一合规 SGX CPU 上运行。\\textbf{S}: `attestation/local.c`:L15-60

[M3.3] Intel Provisioning Certification Service (PCS) 证书分发机制

\\textbf{P:} 平台验证需要确认 QE 签名的 Quote 公钥属于真实的 Intel 芯片。Intel PCS 提供了在线证书吊销与验证服务。\\textbf{ID:} 远端验证服务拉取平台的 PCK（Processor Certificate Key）证书链，Intel 作为根 CA（Root CA）对 PCK 进行签名，从而实现芯片出厂的链式回溯。\\textbf{S:} attestation/pcs.c:L10-45

[M3.4] EPID 群组签名与 ECDSA 认证技术路线演进

\\textbf{P:} 早期 SGX 远程认证采用 EPID (Enhanced Privacy ID) 零知识证明群组签名，隐藏平台唯一物理 ID，保护隐私；SGX3 代后全面转向易于离线验证的 ECDSA 签名体系。\\textbf{ID:} ECDSA 签名由 QE3 飞地生成，配合 PCCS（平台证书缓存服务）能实现秒级的本地证书链解析，避免了频繁远程 Intel 在线 API 调用。\\textbf{S:} attestation/ecdsa.c:L50-110

[M3.5] 测量寄存器（MRENCLAVE 与 MRSIGNER）定义与飞地身份核验

\\textbf{P:} MRENCLAVE 是飞地代码、数据和加载时序的 SHA-256 加密度量值，代表飞地“我是什么”；MRSIGNER 是飞地签名公钥的哈希，代表飞地“谁开发了我”。\\textbf{ID:} 升级飞地代码时，MRENCLAVE 会变，但只要保持签名密钥不变，MRSIGNER 就能保持不变。这允许本地飞地使用 MRSIGNER 策略来共享升级前后的数据封包密钥。\\textbf{S:} sdk/tlibe/tattest.c:L20-55

[M4.1] ECALL 与 OCALL 指令控制流切换的硬件级上下文开销

\\textbf{P:} 进入飞地使用 ECALL（通过 EENTER/EEEXIT 指令组合），飞地调用宿主使用 OCALL。每次 ECALL/OCALL 都涉及页表缓存重置、SSA 压栈和寄存器深度洗牌。\\textbf{ID:} 控制流切换开销约为数千个 CPU 时钟周期。因此，系统设计中应极力避免在循环中触发 ECALL/OCALL，转而使用“无跳转飞地”（Switchless Enclave）线程共享技术。\\textbf{S:} edger8r/codegen.ml:L40-80

[M4.2] 飞地内外数据参数封送（Marshalling）与安全指针检查

\\textbf{P:} 飞地内部的代码不能盲目解引用传递进来的指针（这可能是 OS 的越权陷阱，即 TOCTOU 漏洞）。SGX 自动生成的边界桥接代码必须对参数执行封送拷贝。\\textbf{ID:} SDK 提供的 Edger8r 工具链会自动生成代理代码，检查外部指针所指向的地址是否完全位于飞地外部，如果是，则先将内容深拷贝至飞地受保护内存中，再行操作。\\textbf{S:} edger8r/parser.ml:L100-140

[M4.3] SGX2 EDMM 动态内存管理及 EMA 动态分配机制

\\textbf{P:} 第一代 SGX 要求飞地的 EPC 大小在编译时静态确定，不可动态扩容。SGX2 引入了 EDMM (Enclave Dynamic Memory Management) 硬件机制。\\textbf{ID:} EDMM 支持运行期修改 EPC 页面权限（EMODPR）以及动态增加页面（EMODT）。这允许飞地内部库使用标准的 malloc 从动态扩充的 Heap 中安全分配大内存。\\textbf{S:} sdk/trts/trts_util.cpp:L30-70

[M4.4] 运行时库支持与 LibOS 兼容层设计

\\textbf{P:} 由于 Enclave 内严禁执行系统调用，直接运行复杂应用（如 Redis, Nginx）会导致奔溃。LibOS (Occlum/Gramine) 在 Enclave 内部实现了完整的系统调用翻译层。\\textbf{ID:} LibOS 拦截 libc 的 syscalls，并在 Enclave 内用软件模拟（如内存虚拟文件系统、受保护的网络栈），仅将真正无法处理的底层 I/O 封送并托管给外部普通 OS。\\textbf{S:} sdk/tlibe/string/memset.c:L5-20

[M5.1] Debug Mode 调试特性与 EDGRD/EDWR 指令功能限制

\\textbf{P:} SGX 支持调试模式（Debug Mode）。在 SECS 中，如果设置了 DEBUG 标志位，CPU 将允许使用 EDGRD 和 EDWR 特权指令来读取或修改该飞地的 EPC 物理页面。\\textbf{ID:} 生产模式飞地此标志必须设为 0。一旦关闭调试，即使内核调试器（GDB）通过硬件调试寄存器，也无法抓取到任何 EPC 明文数据，物理 JTAG 调试也会被锁止。\\textbf{S:} sdk/simulation/tcs.cpp:L40-75

[M5.2] 飞地配置文件（XML）及核心安全属性设定

\\textbf{P:} 编译 Enclave 时，通过 XML 文件指定安全属性，如堆栈深度（StackSize）、最大堆大小（HeapSize）、并发线程数（TCSNum）及硬件策略配置。\\textbf{ID:} XML 中的配置属性会被 sgx_sign 工具解析并填充进 SIGSTRUCT 结构的属性字段中，最终由 ECREATE 写入 SECS 并和 MRENCLAVE 度量绑定。\\textbf{S:} sdk/sign_tool/SignTool/SignTool.cpp:L120-170

[M5.3] 飞地签名工具（sgx_sign）双密钥签名方案与 SIGSTRUCT 构成

\\textbf{P:} 飞地签名的目的是生成 SIGSTRUCT 结构体。它包含飞地的测量哈希值、ISVSVN 版本号、开发者公钥，并使用开发者的 RSA-3072 私钥对这部分数据进行数字签名。\\textbf{ID:} 硬件 EINIT 指令会自动解析并验证 SIGSTRUCT 是否包含正确公钥，若被篡改则拒绝启动。这保证了即使用户拿到了物理机器并篡改了 EPC 文件，也无法强行加载。\\textbf{S:} sdk/sign_tool/SignTool/sgx_sign.cpp:L200-280

[M5.4] SGX 模拟模式（Simulation Mode）与硬件真跑的对比边界

\\textbf{P:} 在没有真实 SGX CPU 的平台上，SGX SDK 提供了模拟模式（Simulation Mode），使用软件内存模拟 EPCM 和 AEX 上下文切换。\\textbf{ID:} 模拟模式下不存在 MEE 的物理总线加密，EREPOR 指令计算出 MAC 仅依赖于软件伪造的硬密钥。模拟模式绝对不可用于生产环境，否则形同明文存储。\\textbf{S:} sdk/simulation/assembly/

[M5.5] 架构飞地（Architectural Enclave: LE/QE/PCE/AE）设计作用

\\textbf{P:} 架构飞地是 Intel PSW 提供的一组核心特权 Enclave，包括 LE (Launch Enclave - 分发启动令牌)、QE (Quoting Enclave - 远程认证) 和 PCE (Provisioning Enclave)。\\textbf{ID:} 架构飞地的证书由 Intel Root Key 签发，它们能访问特殊的 CPU Hardware Key（如 Provisioning Key），普通用户飞地无法读取这些密钥，从而实现了特权隔离。\\textbf{S:} psw/ae/common/

[M6.1] EPC 物理内存耗尽时页面换出抖动与调优

\\textbf{P:} 频繁的大内存分配（如对超大数组进行扫描）会导致严重的 EPC 页面换入换出（Paging Thrashing），此时 CPU 处于频繁的重加密和 TLB 刷写状态，性能急剧下降。\\textbf{ID:} 优化手段包括：使用 NUMA 亲和性，避免跨 Socket 的 EPC 访存；减少大数组的全表扫描，设计内存紧凑的数据结构，并在编译 XML 中合理增大初始堆大小。\\textbf{S:} arch/x86/kernel/cpu/sgx/main.c#L450-510

[M6.2] 多飞地间 IPC 通道与安全物理内存共享机制

\\textbf{P:} 两个独立飞地直接的物理内存是完全隔离的（受 EPCM 控制）。为了通信，它们必须协商建立对称会话密钥（Session Key），或者向宿主申请一段非 EPC 的共享内存。\\textbf{ID:} 两个飞地在非 EPC 内存上构建无锁环形队列（Ring Buffer），所有交互报文在飞地内部加密后写入共享内存，由对方飞地读取并解密，确保了宿主 OS 无法窃取通道数据。\\textbf{S:} sdk/attestation/local.c#L80-120

[M6.3] SGX 虚拟化与 KVM vEPC 设备驱动映射

\\textbf{P:} 虚拟机也需要运行 SGX Enclave。KVM 通过虚拟 EPC (vEPC) 设备驱动，将宿主机的一部分物理 EPC 内存直接物理遗传给虚拟机。\\textbf{ID:} KVM 重写了 EPT 页表配置，拦截并重映射虚拟机的 ECREATE 和 EINIT 指令，使得虚拟机内的客户操作系统能够像使用物理 CPU 一样管理 EPC 页面。\\textbf{S:} kvm/vmx/sgx.c:L20-100

[M6.4] 安全版本号（ISVSVN）控制与 TCB 恢复及微码更新审计

\\textbf{P:} ISVSVN 记录了 Enclave 代码的安全版本号。当硬件微架构暴露漏洞时，Intel 会发布微码更新升级 TCB (TCB Recovery)，并更新物理 CPU 的 CPUSVN。\\textbf{ID:} TCB Recovery 后，PCS 会电召并升级 PCK 证书。Enclave 代码必须对比 ISVSVN 和 CPUSVN，强制拒绝在未打安全补丁的旧物理平台上进行密钥派生和初始化。\\textbf{S:} sdk/tlibe/tattest.h:L10-40 \\textbf{—}

Zone T1: Intel SGX Core APIs

sgx_create_enclave() 飞地初始化

sgx_destroy_enclave() 销毁飞地

sgx_ecall()/sgx_ocall() 进出飞地

ECREATE / EADD / EINIT 硬件生命周期

EREPOR / EGETKEY 本地度量证明

Zone T2: Errors & Security Protection

SGX_ERROR_OUT_OF_EPC (EPC 溢出换页抖动)

SGX_ERROR_INVALID_SIGSTRUCT (签名核验失败)

SGX_ERROR_ENCLAVE_LOST (电源切换丢失)

AEX 异常静默寄存器零化

控制通道侧信道防御 (ORAM 平滑访存)